SpotifyClone: Desenvolvimento de uma Aplicação Android com Integração da Spotify Web API

João Teixeira (1211339), Ricardo Barbosa (1211322)

(1211339 e 1211322) @isep.ipp.pt
Licenciatura em Engenharia de Telecomunicações e Informática, Instituto Superior de Engenharia do Porto
Desenvolvimento de Software e Sistemas Móveis (LETI-DSSMV), 2026/27

Abstract. Este artigo descreve o desenvolvimento do SpotifyClone, uma aplicação nativa para Android, implementada em Java, que recorre à Spotify Web API para exploração de catálogo musical e controlo remoto de reprodução num dispositivo Spotify ativo. A solução desenvolvida integra autenticação segura através do protocolo OAuth 2.0 com extensão PKCE, pesquisa de conteúdos, consulta da biblioteca do utilizador, criação e edição de playlists, gestão de músicas favoritas e controlo do estado de reprodução e da fila. Ao longo do documento são descritos o levantamento de requisitos, a modelação em UML e as principais decisões arquiteturais e de implementação, destacando-se a reprodução baseada em contexto para assegurar a construção correta da fila e do modo aleatório (shuffle). A validação do sistema combinou a execução de 315 testes unitários automatizados (isolados da rede) com testes funcionais realizados num dispositivo android físico (em conjunto com o emulador instalado no IDE), confirmando o correto funcionamento das funcionalidades implementadas face às restrições da API externa.

Palavras-chave: Android, Java, Spotify Web API, OAuth 2.0, PKCE, REST, testes unitários.

1 Introdução e Contexto

As aplicações móveis de consumo de conteúdos multimédia assentam na comunicação com serviços externos para autenticação, pesquisa, obtenção de meta dados e controlo de reprodução entre dispositivos. No âmbito da unidade curricular de Desenvolvimento de Software e Sistemas Móveis (LETI-DSSMV), o SpotifyClone foi o projeto desenvolvido com o intuito de explorar essa integração através de uma aplicação Android nativa, desenvolvida em Java, que comunica com a Spotify Web API [1] e apresenta uma interface própria.

O objetivo não foi criar um serviço de distribuição de áudio, mas uma interface móvel que atua sobre a conta Spotify do utilizador e controla remotamente a reprodução. Uma vez que a Web API do Spotify funciona como uma interface de controlo e não fornece transmissão direta de áudio (streaming), a aplicação atua como um controlador remoto: os comandos são enviados para os servidores do Spotify e o áudio é reproduzido no dispositivo oficial que se encontrar ativo (como o computador, web player ou smartphone). Deste modo, o foco técnico do projeto reside na

implementação do fluxo de autenticação segura, no consumo assíncrono de uma API REST e na sincronização contínua da interface gráfica com o estado de reprodução remoto.

O restante documento encontra-se estruturado da seguinte forma: a Secção 2 descreve a análise de requisitos; a Secção 3 apresenta o design do sistema; a Secção 4 detalha a implementação das componentes principais; a Secção 5 expõe a estratégia de testes e os resultados obtidos; e a Secção 6 sintetiza as conclusões e perspetivas de trabalho futuro.

2 Análise e Requisitos

2.1 Requisitos Funcionais

O levantamento de requisitos identificou o utilizador autenticado como único ator do sistema e definiu 15 casos de uso, organizados por domínio funcional na Tabela 1. A estrutura de navegação principal da aplicação divide-se em três secções acessíveis através de uma barra inferior (Início, Pesquisar e Biblioteca), complementadas por um mini-player persistente que permite expandir o leitor em ecrã inteiro.

Tabela 1. Casos de uso da aplicação, agrupados por área.

Área	Casos de uso
Sessão	Iniciar sessão com a conta Spotify (US01)
Exploração	Ver o Início (US02), pesquisar conteúdos (US03), ver um álbum (US11) e um artista (US12)
Biblioteca	Ver playlists e álbuns guardados, com vários modos de visualização (US04)
Playlists	Criar (US05), ver (US06) e editar playlists (US07); adicionar (US08) e remover músicas (US09)
Favoritas	Guardar e remover músicas favoritas (US10)
Reprodução	Tocar música (US13), controlar a reprodução (US14) e gerir a fila (US15)

2.2 Requisitos Não Funcionais e Restrições

Ao nível da segurança, o processo de autenticação não deve armazenar chaves secretas estáticas (Client Secret) no código-fonte, dado que aplicações instaladas no dispositivo do cliente estão sujeitas a engenharia inversa. Adicionalmente, todas as comunicações são efetuadas sobre HTTPS e as chamadas de rede ocorrem de forma assíncrona em threads secundárias, evitando o bloqueio da Main Thread (UI) e garantindo a fluidez da interface, bem como a apresentação de mensagens de erro claras em caso de falha [4].

A solução tira também partido de recursos nativos do sistema Android [4], nomeadamente o serviço ConnectivityManager para verificar o estado da ligação à rede, o seletor de ficheiros do sistema para carregar imagens da galeria aquando da personalização de capas de playlists, e o mecanismo de Deep Links (spotifyclone://callback) para intercetar o redirecionamento final do fluxo OAuth.

Por fim, existem restrições impostas pela própria plataforma do Spotify [1]: os endpoints de controlo de reprodução exigem que o utilizador possua uma subscrição Spotify Premium e um dispositivo de reprodução ativo. Além disso, como a aplicação se encontra registada em modo de desenvolvimento (Development Mode), o acesso à API está circunscrito às contas previamente autorizadas no painel de programador do Spotify.

3 Design

A conceção do sistema foi especificada em UML recorrendo à ferramenta PlantUML. A diretoria docs do repositório reúne os artefactos completos de engenharia de software, incluindo o modelo de domínio, o diagrama de casos de uso, quatro diagramas de classes, bem como os diagramas de sequência de sistema (SSD) e diagramas de sequência detalhados (SD) para cada caso de uso. Nesta secção apresentam-se as vistas mais representativas da arquitetura.

3.1 Modelo de Domínio

A Figura 1 ilustra o modelo de domínio do problema. Um utilizador (User) possui playlists, músicas favoritas, álbuns guardados e um conjunto de dispositivos associados (Device). Um elemento central deste modelo é a entidade PlaybackState, que representa o estado de reprodução mantido remotamente pelos servidores do Spotify, agregando a faixa atualmente em reprodução (Track), a fila de reprodução e o dispositivo ativo onde o áudio está a ser processado.

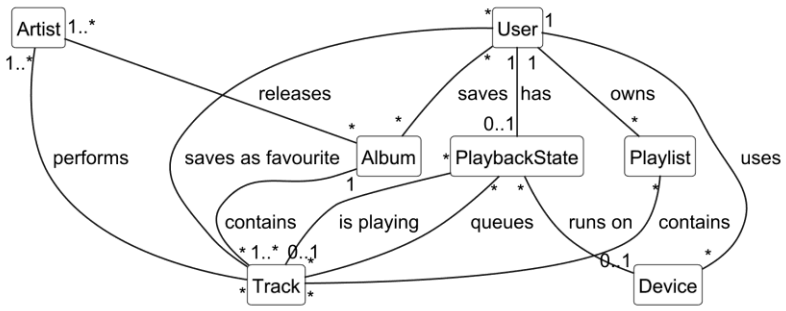


Fig. 1. Modelo de domínio (atributos omitidos; versão completa em `docs/GlobalArtifacts`).

3.2 Arquitetura

A estrutura da aplicação baseia-se em duas atividades principais: a MainActivity, responsável pela gestão da sessão, pela navegação entre ecrãs e pela atualização do mini-player, e a PlayerActivity, dedicada ao controlo detalhado da reprodução em ecrã inteiro. As restantes vistas foram implementadas recorrendo a Fragments alojados na MainActivity. O código-fonte encontra-se modularizado por pacotes segundo a responsabilidade de cada componente, conforme descrito na Tabela 2.

Tabela 2. Organização do código por pacote.

Pacote	Responsabilidade
`fragment`	Ecrãs de Início, Pesquisa, Biblioteca, Playlist, Álbum e Artista
`adapter`	Apresentação de listas em `RecyclerView`
`api`	Pedidos HTTP à Spotify Web API, um cliente por área
`model`	Modelos de dados preenchidos a partir do JSON das respostas
`utils`	Sessão, PKCE, tratamento de respostas e funções auxiliares
`customization`	Cores da barra de navegação e do mini player

Na arquitetura adotada, os fragmentos e atividades comunicam diretamente com as classes clientes da camada api, como se observa na Fig. 2. Esta abordagem permitiu manter a estrutura do projeto concisa e facilitar a leitura do fluxo de dados, embora concentre parte da lógica de controlo nas classes de interface — razão pela qual a transição para o padrão MVVM (com ViewModels e repositórios) é apontada como melhoria futura. Ao nível de bibliotecas externas, utilizou-se o OkHttp para a comunicação HTTP, o Gson para a serialização e desserialização de objetos JSON e o Glide para o carregamento e caching de imagens.

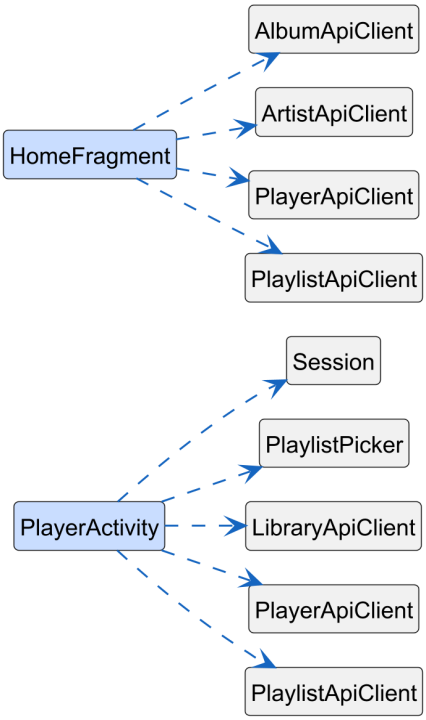


Fig. 2. Excerto das dependências entre ecrãs e clientes da API (diagrama completo em `docs/ClassDiagrams`).

3.3 Comportamento

Enquanto os diagramas SSD representam o sistema como uma caixa negra focada nas interações do utilizador, os diagramas SD especificam a troca de mensagens entre as classes internas da aplicação. A Fig. 3 representa o diagrama de sequência do caso de uso *Tocar Música* (US13) quando iniciado a partir do ecrã de pesquisa. A principal decisão de desenho neste fluxo reside na escolha do método de reprodução: sempre que a faixa selecionada pertence a um álbum com múltiplas músicas, a aplicação instrui a API a reproduzir o contexto do álbum a partir dessa faixa específica (startContextPlaybackAtTrack), garantindo que o Spotify popula automaticamente a fila de reprodução e permite a utilização do modo aleatório.

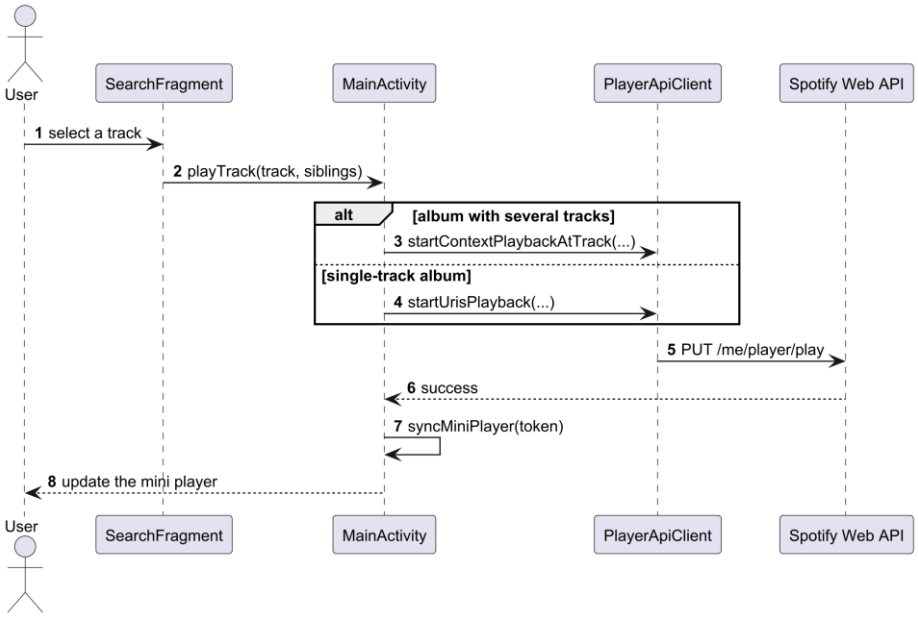


Fig. 3. Diagrama de sequência simplificado do caso de uso Tocar Música (US13).

4 Implementação

4.1 Autenticação e Sessão

O processo de autenticação implementa o fluxo *Authorization Code with PKCE* [2, 3], recomendado para aplicações móveis por não exigir o armazenamento do Client Secret. No início do login é gerado um código aleatório criptograficamente seguro (code_verifier), sendo enviado ao servidor de autorização apenas o seu resumo calculado com o algoritmo SHA-256 e codificado em Base64-URL (code_challenge). O identificador da aplicação (SPOTIFY_CLIENT_ID) é lido a partir do ficheiro local.properties (excluído do controlo de versões), enquanto o armazenamento persistente dos tokens nas SharedPreferences é gerido pela classe Session.

Dado que o access_token atribuído pelo Spotify tem uma validade de uma hora, implementou-se um mecanismo de renovação preventiva através do refresh_token, agendado antes da expiração do token e revalidado sempre que a aplicação regressa ao primeiro plano:

```
private void scheduleTokenRefresh() {
  tokenHandler.removeCallbacks(scheduledTokenRefresh);
  tokenHandler.postDelayed(scheduledTokenRefresh,
      Math.max(0, Session.millisUntilExpired(this)));
}
```

4.2 Comunicação com a API

As chamadas de rede realizadas pelo OkHttp são assíncronas e as respostas chegam numa thread de rede, que não pode alterar a interface. A classe abstrata `ApiCallback` resolve este problema num único ponto: lê a resposta e devolve o resultado à thread principal, separando os casos de sucesso e de erro. Deste modo, as classes da interface apenas necessitam de concretizar o método onSuccess e, opcionalmente, onError.

```
final String finalBody = body;
if (response.isSuccessful()) {
  MAIN.post(() -> onSuccess(finalBody));
} else {
  MAIN.post(() -> onError(code, finalBody));
}
```

4.3 Reprodução e Fila

Numa fase inicial do desenvolvimento, a seleção de uma música num álbum ou nos resultados de pesquisa enviava para o endpoint PUT /me/player/play apenas o URI isolado dessa faixa. Em consequência, o cliente Spotify não dispunha de faixas subsequentes para construir a fila de reprodução, o que impedia o funcionamento correto do modo aleatório (*shuffle*). Para solucionar este comportamento, a reprodução passou a ser iniciada com base no contexto da faixa selecionada:

```
if (track.album != null && track.album.id != null
    && track.album.totalTracks > 1) {
  String albumUri = "spotify:album:" + track.album.id;
  PlayerApiClient.startContextPlaybackAtTrack(token,
      albumUri, track.uriOrBuild(), null, callback);
} else {
  List<String> uris = new ArrayList<>();
  for (Track sibling : siblings) {
    uris.add(sibling.uriOrBuild());
  }
  PlayerApiClient.startUrisPlayback(token, uris,
      siblings.indexOf(track), null, callback);
}
```

Caso a música pertença a um *single* (álbum de faixa única), a aplicação envia a lista de músicas do ecrã atual (siblings) como fila de reprodução. Adicionalmente, na PlayerActivity, a sincronização do estado de reprodução ocorre por *polling* a cada cinco segundos, enquanto um temporizador local incrementa a barra de progresso a cada segundo, assegurando uma atualização visual contínua sem sobrecarregar a API com pedidos.

4.4 Interface

O ecrã de *Início* agrega álbuns, artistas e playlists, reaproveitando os dados obtidos nas mesmas respostas da API para minimizar o número de pedidos HTTP. Na *Biblioteca*, o utilizador pode alternar entre diferentes disposições visuais (lista e grelha), enquanto na PlayerActivity as ações associadas à faixa em reprodução são apresentadas num menu deslizante inferior (*Bottom Sheet*). Visualmente, a aplicação aplica uma imagem de fundo transversal a todos os ecrãs, extraindo dinamicamente as suas cores predominantes para estilizar o *mini-player* e a barra de navegação inferior. Esta lógica foi isolada no pacote customization, desacoplando a estilização dinâmica da lógica dos fragmentos.

5 Testes e Resultados

5.1 Testes Unitários

Os testes unitários usam JUnit 4 e Robolectric [5], permitindo assim executar e testar componentes do SDK Android diretamente na JVM local, sem necessidade de emulador. A estrutura de testes espelha a organização de pacotes da aplicação, totalizando 315 testes unitários aprovados, cuja distribuição se apresenta na Tabela 3.

Tabela 3. Testes unitários por pacote.

Pacote	O que verificam	Testes
`model`	Leitura do JSON e métodos auxiliares	68
`api`	URL, método, cabeçalho e corpo dos pedidos	52
`fragment`	Conteúdo dos ecrãs, navegação e erros	75
`adapter`	Dados das linhas e cliques	27
`utils`	Sessão, PKCE e tratamento de respostas	27
`customization`	Cores aplicadas à interface	5
Atividades	Sessão, navegação, leitor e fila	61
Total		**315**

Para que os testes sejam rápidos, repetíveis e não dependam do serviço real, todos os clientes da API são substituídos por um cliente simulado. Um intercetor do OkHttp regista cada pedido e devolve a resposta definida pelo teste, sem aceder à rede:

```
OkHttpClient fakeClient = new OkHttpClient.Builder()
  .dispatcher(new Dispatcher(new SameThreadExecutor()))
  .addInterceptor(chain -> respond(chain.request()))
  .build();
```

Assim, um teste pode verificar exatamente o que é enviado ao Spotify. O exemplo seguinte garante a correção descrita na Secção 4.3, verificando que, ao reproduzir uma faixa pertencente a um álbum com várias músicas, o corpo JSON enviado inclui o URI do álbum como context_uri e o URI da faixa como offset:

```
@Test
public void playTrack_usesTheAlbumWhenItHasMoreTracks()
    throws IOException {
  MainActivity activity = launchLoggedIn();
  Track track = track(5);

  activity.playTrack(track,
      Collections.singletonList(track));

  assertEquals(
      "{\"context_uri\": \"spotify:album:al1\", "
      + "\"offset\": {\"uri\": \"spotify:track:t1\"}}",
      FakeSpotifyApi.bodyOf(
          api.lastRequestTo(PLAY_PATH)));
}
```

De forma complementar, a geração criptográfica do código de desafio PKCE foi validada comparando o resultado produzido pela classe PKCEUtil com o vetor de teste oficial especificado na norma RFC 7636 [3].

5.2 Testes Funcionais

Os principais fluxos foram testados manualmente com a aplicação ligada a uma conta Spotify real e a música a tocar noutro dispositivo. Os resultados recolhidos constam da Tabela 4, e as respetivas capturas de ecrã das Fig. 4 a 15.

Tabela 4. Testes funcionais e resultados observados.

ID	Caso de teste	Resultado observado	Estado
T01	Abrir o Início (Fig. 4, 5)	Grelha e secções com álbuns, artistas e playlists	Passou
T02	Pesquisar “pop” (Fig. 6, 7)	Géneros substituídos por músicas e artistas	Passou
T03	Ver playlists em lista e grelha (Fig. 8, 9)	Mesmos conteúdos nos dois modos	Passou
T04	Ver álbuns guardados (Fig. 10)	Álbuns com capa, título e artista	Passou
T05	Abrir um artista (Fig. 11)	Foto, nome e discografia com ano e tipo	Passou
T06	Tocar e abrir o leitor (Fig. 12, 13)	Mini player atualizado; capa, progresso, dispositivo e fila	Passou
T07	Guardar e remover das favoritas, com modo aleatório (Fig. 14, 15)	Ícone e mensagem atualizados; modo aleatório ativo	Passou



Fig. 4. Início: grelha e recomendações.

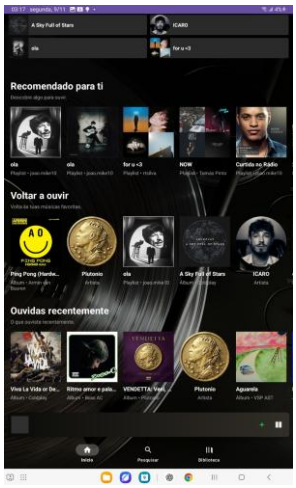


Fig. 5. Início: secção Ouvidas recentemente.

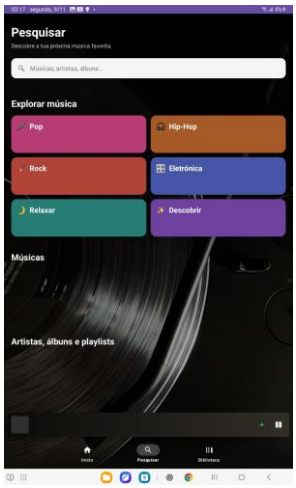


Fig. 6. Pesquisar, antes de introduzir texto.

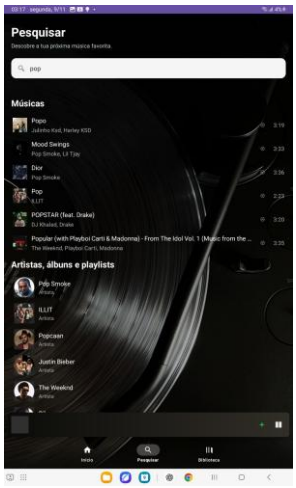


Fig. 7. Resultados da pesquisa "pop".

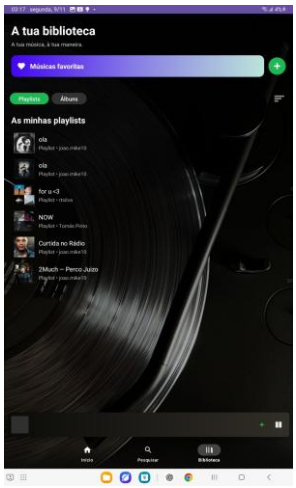


Fig. 8. Biblioteca: playlists em lista.



Fig. 9. Biblioteca: playlists em grelha.



Fig. 10. Biblioteca: álbuns guardados.



Fig. 11. Página de um artista.



Fig. 12. Artista, com música em reprodução.

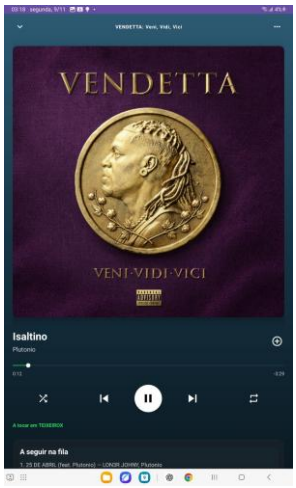


Fig. 13. Leitor com a fila de reprodução.

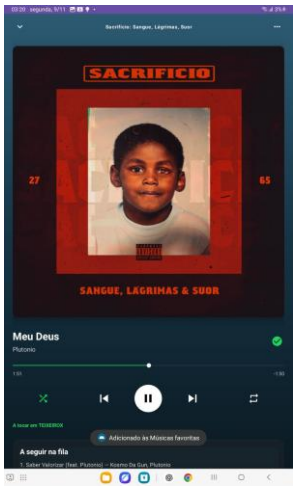


Fig. 14. Música adicionada às favoritas.

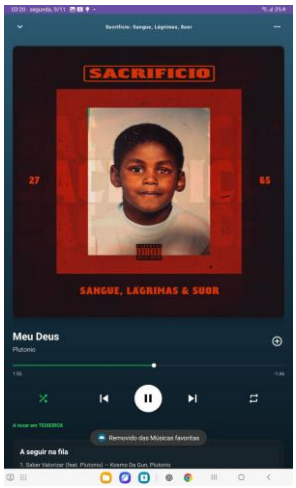


Fig. 15. Música removida das favoritas.

5.3 Limitações

Uma vez que os testes unitários recorrem a respostas simuladas (*mocks*), estes validam a lógica interna da aplicação mas não substituem a verificação face ao serviço real, a qual foi assegurada através dos testes funcionais realizados num dispositivo físico. Ficaram fora da cobertura de testes automatizados a interação com a janela de diálogo de edição de playlists, o carregamento de imagem de capa em Base64 e a paginação automática durante o *scroll*.

Ao nível funcional, a reprodução está dependente da existência de um cliente Spotify ativo na conta do utilizador: quando nenhum dispositivo se encontra acordado, a API devolve um código HTTP 404, situação que é intercetada pela aplicação e comunicada ao utilizador através de um aviso no ecrã. De igual modo, devido às restrições impostas pela Spotify Web API a aplicações em modo de desenvolvimento, a listagem de faixas de algumas playlists geradas pelo Spotify (como *Daily Mixes* ou rádios) devolve erro de permissão (HTTP 403/404), sendo apresentada uma mensagem informativa na interface. Por fim, a secção de discografia na página de artista encontra-se atualmente limitada aos dez primeiros lançamentos sem paginação adicional.

6 Conclusões e Trabalho Futuro

O SpotifyClone demonstra a integração de uma aplicação Android nativa com um serviço externo de música, cobrindo autenticação, consulta de conteúdos, gestão de playlists e controlo da reprodução. O uso de OAuth 2.0 com PKCE e a concentração do tratamento de respostas num único ponto foram as decisões mais importantes de implementação. Os testes foram determinantes: permitiram identificar o problema da fila, que se manifestava no modo aleatório mas tinha origem na forma como a reprodução era iniciada, e os testes unitários protegem agora essa correção.

Como trabalho futuro, propõe-se separar a lógica dos ecrãs com ViewModels e repositórios, guardar os tokens de forma cifrada, guardar em memória as respostas já obtidas, paginar a discografia dos artistas e alargar os testes funcionais a mais dispositivos.

Referências

1. Spotify for Developers: Web API, https://developer.spotify.com/documentation/web-api
2. Spotify for Developers: Authorization Code with PKCE Flow, https://developer.spotify.com/documentation/web-api/tutorials/code-pkce-flow
3. Sakimura, N., Bradley, J., Agarwal, N.: Proof Key for Code Exchange by OAuth Public Clients. RFC 7636, Internet Engineering Task Force (2015)
4. Android Developers: Android Developers Documentation, https://developer.android.com/docs
5. Robolectric: Test-Drive Your Android Code, https://robolectric.org